



**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

**Jakub Portas, Mateusz Skali, Katarzyna Materia,
Magdalena Matuła, Wiktor Kuczek**

Porównanie platform sprzętowych w detekcji anomalii dla
przykładowych zbiorów - Jetson, RPI, ESP32, PC

Bezpieczeństwo sieci komputerowych I

Rzeszów, 2026

Spis treści

1. Wstęp i cel projektu	7
2. Charakterystyka środowisk testowych	9
2.1. Stacja robocza x86_64 (PC) – Środowisko referencyjne	9
2.2. Komputer jednopłytkowy Raspberry Pi 5	9
2.3. NVIDIA Jetson Orin Nano (Edge AI)	10
2.4. Mikrokontroler ESP32 – Warstwa czujnikowa (Deep Edge)	10
3. Analiza i porównanie platform sprzętowych oraz architektur procesorów	11
3.1. Wydajność obliczeniowa i architektura rdzeni	11
3.2. Niezawodność i bezpieczeństwo sprzętowe	12
4. Podstawy teoretyczne wykorzystanych algorytmów detekcji anomalii	14
4.1. Autoencodery	14
4.2. Isolation forest	15
4.3. Half-Space Trees	15
5. Przygotowanie środowiska uruchomieniowego Python na platformie Raspberry Pi	17
6. Przygotowanie środowiska na platformie Jetson Orin Nano	19
7. Problem akceleracji GPU na platformie NVIDIA Jetson	20
7.1. Brak natywnego wsparcia GPU w standardowych bibliotekach w języku Python	20
7.2. Ograniczenia pakietu NVIDIA RAPIDS (cuML) na architekturze ARM64	20
7.3. Konieczność zmiany frameworku dla sieci neuronowych	21
7.4. Próba wdrożenia oficjalnego środowiska kontenerowego (NVIDIA L4T ML)	21
8. Charakterystyka wykorzystanego zbioru danych	23
8.1. Przestrzeń czasowa (Timestamp Data)	23
8.2. Przestrzeń atrybutów fizycznych (Sensor Data)	23
8.3. Zmienna celu	23
9. Metodyka przygotowania danych oraz profilowania wydajnościowego	25
9.1. Akwizycja i preprocessing danych telemetrycznych	25
9.2. Profilowanie termiczne i ograniczenia platformowe	26

9.3. Pomiar czasów globalnych	26
9.4. Analiza opóźnień (Latency) i przepustowości (Throughput)	27
9.5. Profilowanie obciążenia procesora	28
9.6. Obserwacja alokacji pamięci RAM	28
10.Szczegóły implementacyjne i dobór hiperparametrów modeli do	
procesu uczenia	30
10.1. Powtarzalność wyników badań	30
10.2. Implementacja Autoenkodera	30
10.3. Implementacja Isolation Forest	31
10.4. Implementacja Half-Space Trees	31
10.5. Obliczanie progu anomalii	33
11.Proces klasyfikacji i detekcji na zbiorze testowym	34
12.Wyniki detekcji i parametry wydajnościowe	35
13.Podsumowanie i wnioski końcowe	36
Literatura	37

1. Wstęp i cel projektu

W dobie czwartej rewolucji przemysłowej oraz gwałtownego rozwoju systemów Internetu Rzeczy, niezawodne monitorowanie stanu maszyn i wczesne wykrywanie usterek stały się kluczowymi wyzwaniami inżynieryjnymi. Tradycyjne podejście do analizy danych, polegające na przesyłaniu ogromnych ilości surowych odczytów z sensorów do scentralizowanej chmury obliczeniowej, generuje opóźnienia, problemy z przepustowością sieci oraz ryzyka związane z bezpieczeństwem danych. Z tego powodu coraz większą popularność zyskuje koncepcja przetwarzania brzegowego (Edge Computing), która zakłada przeniesienie algorytmów uczenia maszynowego bezpośrednio na urządzenia zlokalizowane blisko źródła sygnału. Wymaga to jednak starannego doboru metod analitycznych do mocno ograniczonych zasobów sprzętowych tych urządzeń.

Wdrożenie przetwarzania brzegowego napotyka jednak na istotny problem inżynieryjny, czyli ogromną fragmentację rynku platform sprzętowych. Współczesny ekosystem Edge obejmuje diametralnie różne architektury od energooszczędnych mikrokontrolerów z niewielką ilością pamięci, przez komputery jednopłytkowe (SBC) ogólnego przeznaczenia oparte na architekturze ARM, aż po wysoce wyspecjalizowane, przemysłowe moduły wyposażone w akceleratory sztucznej inteligencji. Przeniesienie oprogramowania analitycznego ze środowiska stacji roboczej na te urządzenia nierzadko wiąże się ze spadkiem wydajności, problemami z architekturą oprogramowania czy brakiem sprzętowego wsparcia dla określonych instrukcji.

Głównym celem niniejszego projektu grupowego jest wieloaspektowa ewaluacja i porównanie wydajności czterech zróżnicowanych platform sprzętowych w kontekście ich przydatności do zadań przemysłowego uczenia maszynowego. Do testów wytypowano: klasyczny komputer osobisty z procesorem architektury x86_64, nowoczesny komputer jednopłytkowy Raspberry Pi 5, dedykowany moduł Edge w postaci NVIDIA Jetson Orin Nano oraz mikrokontroler z rodziny ESP32. W celu rzetelnego zmierzenia ich możliwości, jako znormalizowane obciążenie testowe (tzw. benchmark) wykorzystano zestaw trzech klasycznych algorytmów detekcji anomalii: Autoenkoder, Isolation Forest oraz strumieniowy model Half-Space Trees. Modele te, różniące się złożonością czasową i zapotrzebowaniem na pamięć operacyjną, posłużyły jako miara do obiektywnego obciążenia badanych architektur procesorów.

Zakres prac koncentruje się przede wszystkim na rygorystycznych testach wydajnościowych sprzętu. Ocenie poddano czas wykonania pełnego potoku danych, opóźnienia (latencję) w przetwarzaniu pojedynczych próbek strumienia oraz maksymalny narzut na zasoby systemowe maszyny. Pomiary jakości samej klasyfikacji pełnią w tym projekcie funkcję walidacyjną. Mają za zadanie udowodnić stabilność i matematyczną tożsamość modeli po ich skompilowaniu na różnych architekturach sprzętowych (ARM vs x86). Kluczowym elementem pracy jest również analiza trudności środowiskowych, takich jak ograniczenia pamięci zunifikowanej, blokady systemowe czy kompatybilność bibliotek języka Python z akceleracją sprzętową. Wyniki tych badań pozwolą na sformułowanie praktycznych wniosków dotyczących doboru optymalnej platformy brzegowej do konkretnych wymagań systemów detekcji anomalii.

2. Charakterystyka środowisk testowych

Aby w pełni zbadać wpływ architektury sprzętowej na wydajność przetwarzania danych, do projektu wytypowano urządzenia reprezentujące cztery skrajnie odmienne środowiska obliczeniowe. Zestawienie to pokrywa pełne spektrum współczesnej inżynierii systemów od scentralizowanych stacji roboczych x86_64, przez uniwersalne komputery jednopłytkowe i wyspecjalizowane moduły akcelеровane sprzętowo, aż po skrajnie ograniczone zasobowo mikrokontrolery. Badane algorytmy posłużyły na każdej z tych platform jako znormalizowane obciążenie testowe, mające na celu weryfikację ich limitów operacyjnych.

2.1. Stacja robocza x86_64 (PC) – Środowisko referencyjne

Jako sprzętowy punkt odniesienia wykorzystano wysokowydajny komputer osobisty Lenovo LOQ. Architektura ta reprezentuje w projekcie klasyczne serwery chmurowe. Wykorzystanie procesora w architekturze x86_64, zasilania sieciowego oraz aktywnego układu chłodzenia eliminuje wszelkie ograniczenia związane z dławieniem termicznym oraz oszczędzaniem energii. Głównym celem implementacji algorytmów na tym urządzeniu było wyznaczenie maksymalnej, teoretycznej przepustowości potoku danych dla użytych bibliotek w języku Python. Wyniki z PC stanowią "złoty standard", pozwalający na precyzyjne zmierzenie, jak duży narzut wydajnościowy w postaci opóźnień lub spadku liczby operacji na sekundę pociąga za sobą przeniesienie kodu na energooszczędne urządzenia brzegowe z rodziny ARM.

2.2. Komputer jednopłytkowy Raspberry Pi 5

W segmencie urządzeń brzegowych ogólnego przeznaczenia badaniom poddano Raspberry Pi 5. W nowoczesnych topologiach Przemysłu 4.0 tego typu minikomputery pełnią zazwyczaj funkcję bramek brzegowych (IoT Gateways), agregując i filtrując dane bezpośrednio z maszyn. Wybór tej platformy podyktowany był chęcią sprawdzenia, jak "surowa", wysokotaktowana architektura ARM Cortex-A76 bez wbudowanych, dedykowanych akceleratorów sztucznej inteligencji radzi sobie ze skomplikowanymi, wielowątkowymi obciążeniami matematycznymi w języku Python. Platforma ta posłużyła do oceny balansu między przystępnością cenową i uniwersalnością sprzętu, a jego realną przepustowością w zadaniach klasycznej analizy danych.

2.3. NVIDIA Jetson Orin Nano (Edge AI)

Trzecim środowiskiem testowym był moduł NVIDIA Jetson Orin Nano. W przeciwieństwie do Raspberry Pi, jest to urządzenie wysoce wyspecjalizowane, zaprojektowane od podstaw dla systemów autonomicznych i przemysłowych, gdzie priorytetem jest niezawodność procesora oraz dostęp do zunifikowanej pamięci operacyjnej współdzielonej między CPU i GPU. Jetson został wybrany do zestawienia jako reprezentant układów ukierunkowanych na sztuczną inteligencję. Ewaluacja na tym urządzeniu miała charakter krytyczny. Jej celem było zbadanie, jak asymetryczna architektura sprzętowa, oparta na wolniej taktowanym, ale bezpiecznym procesorze z rodziny Safety Ready oraz potężnym GPU, reaguje na obciążenie klasycznymi algorytmami CPU-bound jak drzewa decyzyjne z pakietu scikit-learn czy river, których kod źródłowy uniemożliwia wydelegowanie operacji na rdzenie CUDA.

2.4. Mikrokontroler ESP32 – Warstwa czujnikowa (Deep Edge)

3. Analiza i porównanie platform sprzętowych oraz architektur procesorów

Kluczowym elementem oceny wydajności algorytmów na różnych platformach (Edge vs. PC) jest dogłębna analiza ich specyfikacji sprzętowej. Do badań wykorzystano trzy zróżnicowane urządzenia: klasyczny komputer osobisty pełniący rolę punktu odniesienia, wszechstronny komputer jednopłytkowy Raspberry Pi 5 oraz wysoce wyspecjalizowany moduł sztucznej inteligencji NVIDIA Jetson Orin Nano. Zestawienie podstawowych parametrów badanych procesorów przedstawiono w Tabeli 3.1.

Tabela 3.1. Zestawienie parametrów technicznych wykorzystanych platform obliczeniowych

Cecha	Raspberry Pi 5	NVIDIA Jetson Orin Nano	PC (Lenovo LOQ)
Model procesora	Broadcom BCM2712	Niestandardowy układ SoC NVIDIA	AMD Ryzen 7 7435HS
Architektura	Arm Cortex-A76 (64-bit)	Arm Cortex-A78AE (64-bit)	x86_64 (Zen 3+)
Rdzenie / Wątki	4 / 4	6 / 6	8 / 16
Taktowanie	2,4 GHz	1,5 GHz	3,1 GHz (Boost do 4,5 GHz)
Pamięć RAM	4GB LPDDR4X	8GB LPDDR5 (zintegrowana)	Obsługa DDR5 (niezależna)
Pobór mocy (TDP)	Bardzo niski (ok. 5-12W)	Niski (konfigurowalny 7W - 15W)	Wysoki (45W+)

Porównywane platformy sprzętowe reprezentują trzy zupełnie odmienne filozofie projektowania mikroprocesorów, co bezpośrednio determinuje ich potencjał obliczeniowy oraz obszary zastosowań. Podstawowe różnice można sprowadzić do dwóch głównych kategorii: surowej wydajności obliczeniowej oraz mechanizmów zapewniających bezpieczeństwo i niezawodność.

3.1. Wydajność obliczeniowa i architektura rdzeni

Pod względem czystej specyfikacji technicznej najwyższym potencjałem dysponuje procesor AMD Ryzen 7 7435HS. Jest to rozbudowany układ oparty na architekturze x86_64 (mikroarchitektura Zen 3+ Rembrandt R), wyposażony w 8 rdzeni fizycznych z obsługą 16 wątków. Charakteryzuje się on najszybszym zegarem, taktowanie bazowe wynosi 3,1 GHz, a w trybie Boost osiąga 4,5 GHz. Architekturę tę wspiera

niezwykle pojemna hierarchia pamięci podręcznej (16 MB współdzielonej pamięci L3 oraz 4 MB pamięci L2), co predysponuje ten układ do błyskawicznego przetwarzania dużych wolumenów danych.

W segmencie urządzeń wbudowanych (ARM) różnice są równie istotne. Komputer jednopłytkowy Raspberry Pi 5 opiera się na układzie Broadcom BCM2712 z 4-rdzeniowym procesorem Arm Cortex-A76. Dzięki taktowaniu na poziomie 2,4 GHz oraz 2 MB współdzielonej pamięci L3, architektura ta oferuje 2- do 3-krotny wzrost wydajności względem poprzedniej generacji platformy, stanowiąc bardzo wydajną jednostkę ogólnego przeznaczenia.

Z kolei NVIDIA Jetson Orin Nano (w wariantach 8 GB RAM) wykorzystuje 6-rdzeniowy procesor w nowszej architekturze Arm Cortex-A78AE. Choć jego taktowanie wynosi zaledwie 1,5 GHz, sama struktura rdzenia A78AE oferuje o 30

3.2. Niezawodność i bezpieczeństwo sprzętowe

Analizowane układy różnią się drastycznie pod kątem mechanizmów bezpieczeństwa, co wynika z ich rynkowego przeznaczenia:

- a) **Cortex-A78AE (NVIDIA Jetson)** – Jest to architektura zbudowana wokół koncepcji "Safety Ready", dedykowana do systemów autonomicznych, robotyki i przemysłu ciężkiego. Jej najważniejszą cechą jest sprzętowa obsługa funkcji Split-Lock. Pozwala ona rdzeniom na pracę w trybie wysokiej niez. W tym trybie dwa rdzenie wykonują synchronicznie tę samą operację, wzajemnie weryfikując swoje wyniki, co pozwala na natychmiastowe wykrycie i wyeliminowanie błędów obliczeniowych (np. wywołanych promieniowaniem kosmicznym)
- b) **Zen 3+ (AMD Ryzen)** – Architektura ta skupia się przede wszystkim na stabilności i inteligentnym zarządzaniu energią w ekosystemach serwerowych i konsumenckich, wprowadzając ponad 50 nowych stanów zasilania względem poprzedniej generacji.
- c) **Cortex-A76 (Raspberry Pi 5)** – Oferuje sprzętowe rozszerzenia kryptograficzne, które znacząco przyspieszają szyfrowanie strumieni danych (np. SSL/TLS). Stanowi to doskonałe zabezpieczenie dla standardowych aplikacji sieciowych, jednak sam procesor nie posiada certyfikacji ani mechanizmów nadmiarowych

(jak Jetson) pozwalających na użycie go w krytycznych systemach podtrzymujących funkcje życiowe lub sterujących autonomicznymi pojazdami.

4. Podstawy teoretyczne wykorzystanych algorytmów detekcji anomalii

W ramach niniejszego projektu do detekcji anomalii wykorzystano trzy algorytmy reprezentujące odmienne paradygmaty uczenia maszynowego: sieci neuronowe oparte na rekonstrukcji sygnału, klasyczne metody zespołowe oparte na drzewach decyzyjnych oraz algorytmy strumieniowe.

4.1. Autoencodery

Autoenkoder to specyficzny rodzaj sztucznej sieci neuronowej, którego celem jest rekonstrukcja danych wejściowych na warstwie wyjściowej. Sieć ta składa się z dwóch głównych części: enkodera, który kompresuje dane wielowymiarowe do ukrytej reprezentacji o niższym wymiarze, tzw. wąskiego gardła, oraz dekodera, który stara się odtworzyć pierwotny sygnał na podstawie tej skompresowanej reprezentacji. W kontekście detekcji anomalii, model ten jest trenowany wyłącznie na danych opisujących normalne działanie systemu. W rezultacie sieć uczy się ignorować szum i poprawnie rekonstruować powtarzalne wzorce. Gdy na wejściu sieci pojawi się anomalia, której model wcześniej nie widział, enkoder nie potrafi poprawnie zmapować go w swojej warstwie ukrytej, co skutkuje wysokim błędem rekonstrukcji, najczęściej mierzonym jako błąd średniokwadratowy MSE (4.1). Obserwacje przekraczające wyznaczony próg błędu są klasyfikowane jako anomalie.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (4.1)$$

Jak wykazali Chandola i in. w klasycznym przeglądzie literatury dotyczącym detekcji anomalii (Anomaly Detection: A Survey, 2009), modele oparte na rekonstrukcji są niezwykle skuteczne w przypadku danych wielowymiarowych. Znajdują one szerokie zastosowanie w analizie sygnałów z wielu zintegrowanych sensorów przemysłowych, gdzie kluczowe jest wychwycenie skomplikowanych, nieliniowych zależności pomiędzy poszczególnymi czujnikami, które są trudne do zamodelowania tradycyjnymi metodami statystycznymi.

4.2. Isolation forest

Algorytm Lasu Izolacyjnego różni się fundamentalnie od większości metod detekcji, które starają się najpierw zdefiniować profil normalności, a następnie szukają odchyłeń od tej normy. Metoda ta skupia się na bezpośrednim odizolowaniu anomalii. Algorytm buduje zespół losowych drzew binarnych. W każdym kroku losowo wybierana jest cecha, np. odczyt z konkretnego czujnika, a następnie losowo dobierany jest punkt podziału pomiędzy minimalną a maksymalną wartością tej cechy. Z racji tego, że anomalie są z definicji nieliczne i odmienne, wymagają one znacznie mniejszej liczby podziałów, aby zostać odizolowanymi od reszty zbioru danych w pojedynczym liściu drzewa. W związku z tym, ścieżka od korzenia do liścia dla anomalii jest znacznie krótsza niż dla normalnych próbek. Ocena anomalii dla każdego punktu danych jest wyliczana na podstawie średniej długości ścieżki we wszystkich wygenerowanych drzewach.

Zaproponowany przez autorów Liu, Tinga i Zhou (Isolation Forest, 2008) algorytm charakteryzuje się liniową złożonością czasową oraz niskim zapotrzebowaniem na pamięć operacyjną. Ponieważ nie opiera się na obliczaniu odległości czy gęstości, świetnie sprawdza się w wielowymiarowych zbiorach danych. Jest szeroko stosowany w cyberbezpieczeństwie oraz diagnostyce maszyn przy przetwarzaniu wsadowym, będąc wysoce odpornym na szum w danych treningowych.

4.3. Half-Space Trees

Algorytm Half-Space Trees to nowsze podejście oparte na strukturach drzewiastych, jednak zaprojektowane od podstaw do pracy w środowisku strumieniowym. W przeciwieństwie do standardowych algorytmów wsadowych, które wymagają wczytania całego zbioru danych do pamięci, HS-Trees uczy się inkrementalnie próbka po próbce. Algorytm tworzy zbiór drzew losowych, gdzie każda przestrzeń cech jest dzielona na równe półprzestrzenie, stąd nazwa Half-Space. Przechodzące przez system dane aktualizują liczniki w węzłach drzew. Model ten często wykorzystuje mechanizm okna czasowego, dzięki czemu starsze dane są zapominane, a algorytm adaptuje się do zmieniającego się w czasie profilu normalnego zachowania maszyny. Próbka, która trafia do węzła o bardzo niskim liczniku wystąpień w danym oknie czasowym, otrzymuje wysoką punktację anomalii.

Jak udowodniono w pracy twórców algorytmu (Tan, Ting i Liu, Fast Anomaly Detection for Streaming Data, 2011), HS-Trees to rozwiązanie dedykowane dla środowisk, w których dane napływają w sposób ciągły z dużą prędkością, a zasoby obliczeniowe i pamięciowe są ograniczone. Ze względu na swój stały czas aktualizacji i minimalny odcisk w pamięci RAM, algorytm ten jest idealnym kandydatem do zastosowań w ekosystemach Internetu Rzeczy oraz platformach brzegowych, gdzie urządzenia takie jak Raspberry Pi muszą dokonywać predykcji w czasie rzeczywistym na bezpośrednich odczytach z maszyn.

5. Przygotowanie środowiska uruchomieniowego Python na platformie Raspberry Pi

Podczas konfiguracji platformy Raspberry Pi napotkano na trudności związane z instalacją bibliotek projektowych za pomocą globalnego menedżera pakietów pip. Próby te kończyły się krytycznym błędem informującym o "środowisku zarządzanym zewnętrznym", co jest bezpośrednim wynikiem rygorystycznego wdrożenia standardu PEP 668 w najnowszych systemach operacyjnych opartych na architekturze Debian. Standard ten wprowadzono w celu eliminacji konfliktów zależności pomiędzy systemowym instalatorem apt, a menedżerem pakietów języka Python. Ponieważ współczesne dystrybucje Linuksa opierają stabilność swoich wewnętrznych usług na konkretnych wersjach bibliotek Pythona, modyfikacja globalnej przestrzeni nazw przez użytkownika mogłaby doprowadzić do awarii całego systemu. W odpowiedzi na wykrycie znacznika EXTERNALLY-MANAGED w systemowym katalogu, narzędzie pip celowo blokuje instalację, wymuszając tym samym stosowanie bezpieczniejszych praktyk inżynierii oprogramowania.

Aby zainstalować zależności wymagane przez algorytmy uczenia maszynowego bez ryzyka naruszenia stabilności powłoki systemu, konieczne było utworzenie odizolowanego środowiska projektowego. W tym celu wykorzystano wbudowany w język Python moduł venv. W pierwszej kolejności, w docelowym katalogu projektu, wygenerowano wirtualne środowisko uruchomieniowe przy użyciu polecenia 5.1.

Listing 5.1. Inicjalizacja środowiska venv

```
python -m venv .venv
```

Komenda ta utworzyła wydzieloną strukturę katalogów, zawierającą niezależną kopię interpretera języka Python oraz własną, pustą przestrzeń na pakiety. Następnie, aby skierować system do korzystania z tej wyizolowanej instancji, środowisko zostało aktywowane komendą 5.2.

Listing 5.2. Włączenie środowiska venv

```
source .venv/bin/activate
```

Przejsie w tryb środowiska wirtualnego, sygnalizowane pojawieniem się prefiksu .venv w wierszu poleceń, odblokowało pełny, bezpieczny dostęp do menedżera pakietów. Dopiero w tak odizolowanym kontenerze możliwe było pobranie i zainstalo-

wanie wszystkich niezbędnych bibliotek za pomocą polecenia `pip install`. Zastosowana procedura nie tylko pozwoliła na sprawne ominięcie blokad nałożonych przez system operacyjny, ale również zagwarantowała, że specyficzne wersje pakietów użytych do trenowania modeli detekcji anomalii pozostaną w pełni hermetyczne i nie wpłyną negatywnie na działanie innych usług działających na urządzeniu Raspberry Pi.

6. Przygotowanie środowiska na platformie Jetson Orin Nano

Początkowy etap prac projektowych obejmował przygotowanie odpowiedniego środowiska operacyjnego na module NVIDIA Jetson Orin Nano, co wiązało się z nieoczekiwanymi trudnościami sprzętowymi. Pierwsza próba polegała na wykorzystaniu posiadanego dysku z zainstalowanym systemem Kali Linux. Próba ta zakończyła się natychmiastowym niepowodzeniem ze względu na fundamentalne różnice w architekturze procesorów. Powszechnie stosowane systemy operacyjne z komputerów osobistych są kompilowane pod architekturę x86_64, która jest całkowicie niekompatybilna z układami wbudowanymi firmy NVIDIA, opartymi na architekturze ARM64. Moduł Jetson nie był w stanie rozpoznać ani uruchomić instrukcji z takiego nośnika.

Wobec powyższego, dysk został sformatowany w celu przeprowadzenia czystej instalacji dedykowanego systemu. Podjęto próbę klasycznego zbootowania instalatora z nośnika USB bezpośrednio na przygotowany dysk docelowy. To podejście również zakończyło się fiaskiem, uwydatniając kolejną cechę charakterystyczną platform brzegowych. Urządzenia z rodziny Jetson nie posiadają klasycznego układu BIOS/UEFI znanego z komputerów PC, który pozwalałby na swobodne bootowanie z zewnętrznych urządzeń USB. Ich proces rozruchu opiera się na specyficznym bootloaderze, który do inicjalizacji systemu na zewnętrznych dyskach NVMe/SSD wymaga przeprowadzenia skomplikowanego procesu flashingu za pomocą zewnętrznego komputera-hosta i narzędzia NVIDIA SDK Manager.

Ostatecznie, aby uzyskać stabilne i działające środowisko w sposób optymalny czasowo, konieczna była zmiana nośnika pamięci. Zgodnie z natywnym wsparciem rozruchowym modułu, zakupiono kartę pamięci microSD, na którą wgrano dedykowany, oficjalny obraz systemu operacyjnego Linux for Tegra JetPack OS. Wykorzystanie wbudowanego interfejsu kart pamięci pozwoliło na natychmiastowe i bezproblemowe zbootowanie systemu. Dopiero ten krok umożliwił przejście do właściwych prac związanych z konfiguracją środowiska programistycznego i testowaniem algorytmów uczenia maszynowego.

7. Problem akceleracji GPU na platformie NVIDIA Jetson

Podczas prób wdrożenia i przyspieszenia algorytmów detekcji anomalii na platformie NVIDIA Jetson Orin Nano, napotkano barierę programową. Pomimo fizycznej obecności wydajnego układu graficznego, żadna z wykorzystywanych bibliotek nie była w stanie wydelegować do niego obliczeń. Analiza tego zjawiska wykazała, że problem wynika bezpośrednio z braku kompatybilności ekosystemu klasycznego uczenia maszynowego z architekturą CUDA w środowisku ARM64. Ograniczenia te można podzielić na cztery główne kategorie.

7.1. Brak natywnego wsparcia GPU w standardowych bibliotekach w języku Python

Podstawowe biblioteki użyte w projekcie, takie jak scikit-learn, odpowiedzialna za modele Isolation Forest oraz MLPRegressor oraz river, implementująca algorytm Half-Space Trees, zostały zaprojektowane od podstaw z myślą o wykonywaniu obliczeń wyłącznie na głównym procesorze. Ich kod źródłowy nie posiada zaimplementowanych tzw. backendów dla technologii NVIDIA CUDA. Oznacza to, że niezależnie od konfiguracji sprzętowej urządzenia, biblioteki te fizycznie nie potrafią skompilować swojego grafu obliczeniowego do instrukcji zrozumiałych dla rdzeni graficznych i zawsze wymuszają wykonanie kodu na procesorze głównym.

7.2. Ograniczenia pakietu NVIDIA RAPIDS (cuML) na architekturze ARM64

Rozwiązaniem problemu braku wsparcia GPU dla środowiska PC jest zazwyczaj użycie dedykowanej biblioteki NVIDIA RAPIDS (cuML). Niestety, platforma Jetson opiera się na architekturze ARM64, podczas gdy pakiety RAPIDS są rozwijane, kompilowane i optymalizowane głównie pod kątem dużych serwerów opartych na architekturze x86_64. Próba instalacji tych bibliotek za pomocą standardowych menedżerów pakietów, np. pip na Jetsonie kończy się niepowodzeniem z powodu braku gotowych, prekompilowanych plików binarnych. Wymusza to niezwykle skomplikowaną kompilację ze źródeł, która na urządzeniach brzegowych często skutkuje konfliktami wersji kompilatora lub niekompatybilnością z wersją oprogramowania JetPack.

7.3. Konieczność zmiany frameworku dla sieci neuronowych

W przypadku algorytmu Autoenkodera, realizowanego za pomocą klasy MLPRegressor z biblioteki scikit-learn, ograniczenie jest identyczne, biblioteka ta nie potrafi korzystać z akceleracji sprzętowej. Aby wykorzystać potencjał modułu Jetson Orin Nano do sieci neuronowych, konieczne byłoby całkowite przepisanie architektury modelu z użyciem frameworków takich jak PyTorch lub TensorFlow. Tylko te potężne biblioteki Deep Learningowe posiadają pełne, natywne wsparcie dla ekosystemu JetPack oraz potrafią bezpośrednio mapować operacje na rdzenie CUDA i Tensor w architekturze ARM.

7.4. Próba wdrożenia oficjalnego środowiska kontenerowego (NVIDIA L4T ML)

W ramach pogłębionej diagnostyki i wykluczenia ewentualnych błędów konfiguracyjnych systemu operacyjnego, podjęto próbę uruchomienia skryptów wewnątrz oficjalnego kontenera Docker, udostępnianego przez firmę NVIDIA dla platform brzegowych, z rodziny Linux for Tegra Machine Learning. Chociaż to wysoce zoptymalizowane środowisko dostarcza preinstalowane sterowniki CUDA, cuDNN oraz TensorRT, nie przyniosło ono oczekiwanego rozwiązania w postaci akceleracji GPU dla badanych algorytmów. Analiza zawartości kontenera wykazała, że oficjalne obrazy NVIDIA są przygotowywane niemal wyłącznie pod kątem wsparcia głębokich sieci neuronowych. Wchodzące w skład kontenera biblioteki do klasycznej analizy danych, takie jak scikit-learn czy pakiety strumieniowe, to wciąż ich standardowe, procesorowe dystrybucje. Środowisko kontenerowe, nawet z poprawnie zmapowanym akceleratorem graficznym, nie potrafi automatycznie przetłumaczyć ani wymusić wykonania instrukcji sprzętowych dla algorytmów, które architektonicznie nie posiadają zaprogramowanego backendu CUDA. Eksperyment ten ostatecznie potwierdził, że brak akceleracji wynika bezpośrednio z braku implementacji algorytmów na GPU w ekosystemie ARM64, a nie z błędnej konfiguracji samego urządzenia Jetson.

Zjawisko braku możliwości wykonania algorytmów na GPU udowadnia, że przeniesienie projektu na platformę brzegową wymaga ścisłego doboru technologii oprogramowania. Klasyczne uczenie maszynowe w języku Python pozostaje w dużej mierze domeną procesorów. Próba wymuszenia akceleracji sprzętowej na platformie ARM dla

algorytmów niebędących głębokimi sieciami neuronowymi napotyka na nieprzewyżnione w standardowy sposób bariery w samym kodzie źródłowym dostępnych bibliotek.

8. Charakterystyka wykorzystanego zbioru danych

Do celów badawczych, treningu oraz ewaluacji zaimplementowanych algorytmów wykorzystano ogólnodostępny zbiór danych Pump Sensor Data udostępniony na platformie Kaggle. Reprezentuje on rzeczywiste, historyczne odczyty parametrów pracy przemysłowej pompy wodnej zlokalizowanej w niewielkiej miejscowości. Zbiór ten stanowi klasyczny przykład danych typu Internet of Things pochodzących z systemów SCADA, co czyni go idealnym fundamentem do symulacji obciążeń na platformach brzegowych. Struktura zbioru podzielona jest na trzy główne grupy informacyjne.

8.1. Przestrzeń czasowa (Timestamp Data)

Zbiór określa dokładny moment wykonania każdego pomiaru. Odczyty ze wszystkich czujników były rejestrowane synchronicznie w stałych, jednominutowych interwałach. Taka ziarnistość pozwala na traktowanie zgromadzonych informacji jako wielowymiarowych szeregów czasowych. Dzięki precyzyjnym znacznikom czasowym możliwe jest sekwencyjne odtwarzanie danych w pętli programu, symulując strumieniowanie w czasie rzeczywistym dla algorytmu Half-Space Trees oraz prowadzenie analizy trendów środowiskowych poprzedzających wystąpienie awarii.

8.2. Przestrzeń atrybutów fizycznych (Sensor Data)

Główną część zbioru stanowią surowe odczyty fizyczne z urządzenia. Baza zawiera 52 niezależne serie danych pochodzące z oddzielnych jednostek sensorowych maszyny, oznaczonych odpowiednio od `sensor_00` do `sensor_51`. Ponieważ są to wartości surowe, prosto z systemów pomiarowych, charakteryzują się one obecnością naturalnego szumu pomiarowego, wartości odstających oraz brakami w danych. Skrajnym przypadkiem jest zmienna `sensor_15`, która składa się całkowicie z wartości pustych. Taka struktura bazy wymusiła w początkowym etapie przeprowadzania badań zastosowanie mechanizmów wstępnego przetwarzania danych, w tym imputacji braków metodami propagacji wartości oraz standaryzacji zakresów za pomocą algorytmu `MinMaxScaler`.

8.3. Zmienna celu

Kluczowym atrybutem zbioru, pełniącym funkcję referencyjną, tzw. `ground truth`, jest etykieta aktualnego stanu maszyny. Choć ewaluowane algorytmy należą

do rodziny modeli nienadzorowanych lub częściowo nadzorowanych, etykieta ta była niezbędna do końcowej oceny ich skuteczności (wyliczenia macierzy pomyłek i metryki F1-Score). Występuje ona w trzech stanach:

- a) **NORMAL (Praca normalna)** – Oznacza poprawne funkcjonowanie pompy. W ramach metodyki projektowej, podzbiór danych oznaczony tym statusem został odizolowany i posłużył jako jedyne źródło wiedzy dla modeli w fazie treningu (uczenie się zdrowego profilu maszyny).
- b) **BROKEN (Awaria)** – Oznacza krytyczny moment wystąpienia usterki systemu. W analizowanym rocznym okresie badawczym odnotowano dokładnie 7 takich zdarzeń. Stanowią one główne anomalie, które implementowane modele miały za zadanie wykryć wyłącznie na podstawie odchyłeń od normy w odczytach z sensorów.
- c) **RECOVERING (Okres regeneracji)** – Wskazuje na okres przejściowy następujący po awarii, w którym system pompowy był naprawiany, stabilizowany i przywracany do pełnej sprawności operacyjnej.

Zastosowanie tak zdefiniowanego, surowego i wysoce niebalansowanego zbioru danych pozwoliło na przeprowadzenie realistycznych testów zaimplementowanych architektur detekcji anomalii, wiernie oddając warunki panujące w nowoczesnym przemyśle.

9. Metodyka przygotowania danych oraz profilowania wydajnościowego

9.1. Akwizycja i preprocessing danych telemetrycznych

Eksperymenty przeprowadzono z wykorzystaniem zbioru Pump sensor data, zawierającego wielowymiarowe szeregi czasowe z 52 czujników przemysłowych, rejestrowane z częstotliwością jednej minuty. W pierwszej fazie preprocessingu dokonano selekcji cech, odrzucając metadane niebędące odczytami fizycznymi (znacznik czasu, indeks, tekstowa etykieta statusu).

Ze względu na niekompletność strumienia danych (szczególnie zauważalną w przypadku sensora nr 15), zastosowano kaskadową strategię imputacji braków. W pierwszej kolejności wykorzystano metodę propagacji ostatniej znanej wartości w przód (ang. forward fill), następnie w tył (backward fill), a wszelkie rezydualne braki zastąpiono zerami.

Proces binaryzacji etykiet przekształcił problem w zadanie klasyfikacji dwuklasowej:

- Stan NORMAL (poprawna praca układu) zamapowano jako 0,
- Stany awaryjne BROKEN oraz przejściowe RECOVERING zagregowano do wspólnej klasy anomalii oznaczonej jako 1.

Podziału na zbiór treningowy (70%) i testowy (30%) dokonano z rygorystycznym zachowaniem chronologii szeregu czasowego. Wyłączenie tasowania próbek było krytyczne, aby zapobiec wyciekowi danych z przyszłości do zbioru uczącego. Indeksy podziału utrwalono w plikach train.txt i test.txt, gwarantując pełną reprodukowalność eksperymentów dla różnych algorytmów.

Następnie surowe odczyty poddano normalizacji za pomocą transformacji Min-MaxScaler, skalując przestrzeń cech do przedziału $[0, 1]$. Standaryzacja ta jest niezbędnym warunkiem stabilnej konwergencji sieci neuronowych (Autoenkoderów). W ostatnim kroku, bazując na paradygmacie uczenia nienadzorowanego, na zbiór treningowy nałożono maskę logiczną, filtrując wyłącznie odczyty zdrowej maszyny. Dzięki temu algorytmy modelowały wyłącznie profil "normy", kwalifikując wszelkie odchylenia w fazie testowej jako anomalię.

9.2. Profilowanie termiczne i ograniczenia platformowe

Istotnym elementem oceny wydajności modeli na urządzeniach brzegowych było monitorowanie temperatury układów obliczeniowych. Próba ujednoliconego pomiaru termicznego za pomocą skryptu napisanego w języku Python uwypukliła jednak wyzwania związane z fragmentacją środowisk sprzętowych i architekturą systemów operacyjnych. W toku eksperymentów ustalono, że stabilne i wiarygodne profilowanie temperatury udało się zrealizować wyłącznie na jednej z czterech badanych platform. Ograniczenia wynikały z następujących różnic architektonicznych:

- a) **ESP32**
- b) **Komputer osobisty oraz Nvidia Jetson** – Na obu tych platformach próby odczytu temperatury z poziomu skryptu zakończyły się niepowodzeniem. Wynikało to ze specyfiki systemów operacyjnych, zabezpieczeń izolujących procesy użytkownika oraz braku ustandaryzowanego interfejsu takiego jak np. uniwersalny plik wirtualny w systemach Linux. Skrypt w języku Python nie był w stanie uzyskać bezpośredniego dostępu do telemetrii sensorów termicznych na płycie głównej bez głębokiej ingerencji w uprawnienia administracyjne (np. wywoływania zewnętrznych narzędzi systemowych typu `lm-sensors`, co zaburzałoby uniwersalność kodu).
- c) **Raspberry Pi** – Architektura systemu operacyjnego tego mikrokomputera okazała się w pełni kompatybilna z przyjętym modelem pomiarowym. Platforma ta jako jedyna pozwoliła na stabilny i ciągły odczyt temperatury układu, co zrealizowano poprzez bezpośrednie odczytywanie wartości z systemowego pliku wirtualnego `/sys/class/thermal/thermal_zone0/temp`. Zebrane w ten sposób dane umożliwiły precyzyjną ocenę nagrzewania się urządzenia pod obciążeniem algorytmów analitycznych.

9.3. Pomiar czasów globalnych

W tym trybie wykorzystano standardową funkcję systemową `time.time()`, która rejestruje bezwzględny czas wykonywania się głównych bloków kodu. Funkcję wywoływano (oraz obliczano różnicę czasu) w następujących momentach działania aplikacji:

- a) **Czas przygotowania danych** – Wyznaczenie czasu potrzebnego na załadowanie zbioru CSV do pamięci, uzupełnienie braków (tzw. kaskadowy fill) oraz skalowanie cech.
- b) **Czas treningu (t_{train})** – Znaczniki czasu wstawiono bezpośrednio przed i po bloku uczącym dany model, co pozwoliło określić globalny czas potrzebny na wytworzenie profilu normalnego zachowania na konkretnej platformie.
- c) **Całkowity czas detekcji ($t_{\text{detect_total}}$)** – Pomiar ten obejmował przepuszczenie całego zbioru testowego przez nauczony model naraz, co symuluje wydajność klasycznego przetwarzania paczkowego (ang. batch processing).

9.4. Analiza opóźnień (Latency) i przepustowości (Throughput)

Kluczowym kryterium ewaluacji systemów wbudowanych jest czas reakcji na nowe zdarzenia. W celu rzetelnego zmierzenia opóźnień wnioskowania (ang. inference latency), ze zbioru testowego wyizolowano losową próbę 200 obserwacji.

Oprogramowanie przetwarzało te próbki sekwencyjnie w pętli (symulując napływający strumień danych z czujników), rejestrując czas z mikrosekundową precyzją bezpośrednio przed i po wywołaniu predykcji modelu. Zebrane czasy posłużyły do wyznaczenia kluczowych metryk brzegowych:

- a) **Średnia latencja (lat_mean)** – Przeciętny czas, jakiego model potrzebuje na przeanalizowanie jednego wektora cech (odpowiadającego jednej minucie pracy maszyny).
- b) **Kwantyle opóźnień (lat_p95 , lat_p99)** – Wartości określające górną granicę czasu potrzebnego na przetworzenie odpowiednio 95% i 99% najgorszych (najdłużej analizowanych) próbek. Metryka P99 jest w inżynierii standardem określającym stabilność i gwarancję czasu odpowiedzi (SLA) w systemach czasu rzeczywistego.
- c) **Przepustowość (throughput)** – Wyrażona w liczbie przetworzonych próbek na sekundę, określa maksymalną wydajność systemu. Informuje, ile minut historycznej lub napływającej telemetrii dane urządzenie jest w stanie przeanalizować w ciągu jednej sekundy czasu rzeczywistego.

9.5. Profilowanie obciążenia procesora

Do monitorowania aktywności jednostki obliczeniowej wykorzystano zewnętrzną bibliotekę `psutil`, skonfigurowaną do pracy w trybie różnicowym (z parametrem `interval=None`). Podejście to pozwala na asynchroniczne badanie średniego obciążenia procesora pomiędzy zdefiniowanymi punktami kontrolnymi w kodzie, bez sztucznego wstrzymywania i blokowania głównego wątku aplikacji. Cykl zbierania metryk podzielono na trzy strategiczne etapy:

- a) **Inicjalizacja punktu odniesienia** – Pierwsze wywołanie funkcji na starcie skryptu, pełniące rolę kalibracji systemu (uruchomienie wewnętrznego timera biblioteki `psutil`).
- b) **Odczyt po przygotowaniu danych** – Rejestracja narzutu obliczeniowego wygenerowanego przez operacje wejścia/wyjścia (I/O), proces kaskadowego uzupełniania braków oraz normalizację cech.
- c) **Odczyt po sfinalizowaniu treningu** – Ewaluacja kosztu zasobów zużytych stricte na optymalizację algorytmu (np. budowę drzew binarnych lub dopasowanie wag w sieci neuronowej).

Końcowy wskaźnik utylizacji procesora, prezentowany w raporcie podsumowującym, wyznaczany jest jako średnia arytmetyczna z pomiarów zarejestrowanych w etapach drugim i trzecim. Przyjęta metodyka gwarantuje miarodajną i obiektywną ocenę całkowitego kosztu obliczeniowego, jakiego wymagała kompleksowa realizacja głównej logiki detekcyjnej systemu.

9.6. Obserwacja alokacji pamięci RAM

Pomiary zużycia pamięci operacyjnej zrealizowano przy użyciu wbudowanej biblioteki `tracemalloc`, która umożliwia precyzyjne śledzenie alokacji pamięci przez procesy środowiska Python. Proces monitorowania inicjowany jest wywołaniem funkcji `tracemalloc.start()` bezpośrednio przed rozpoczęciem fazy treningowej modelu. Od tego momentu system rejestruje każdy zaalokowany blok pamięci, pozwalając na dokładne uchwycenie rzeczywistego narzutu pamięciowego algorytmu w trakcie jego pracy.

Po sfinalizowaniu procesu uczenia wywoływana jest metoda `tracemalloc.get_traced_memory()`. Funkcja ta zwraca krotkę zawierającą dwie zmienne: zużycie bieżące oraz zużycie szczy-

towe. Do celów analitycznych ekstrahowana jest wyłącznie wartość szczytowa, ponieważ to ona determinuje maksymalne zapotrzebowanie modelu na zasoby, co stanowi parametr krytyczny przy wdrażaniu rozwiązań na urządzenia brzegowe o ograniczonej pojemności pamięci SRAM/RAM. Uzyskany wynik, domyślnie wyrażony w bajtach, poddawany jest konwersji na megabajty poprzez podzielenie przez 1024^2 . Cykl profilowania kończy się wywołaniem `tracemalloc.stop()`, co skutecznie czyści bufor monitorujące i zwalnia zasoby systemowe przypisane do analizatora.

10. Szczegóły implementacyjne i dobór hiperparametrów modeli do procesu uczenia

Zrozumienie działania badanych algorytmów wymaga analizy ich implementacji w kodzie oraz uzasadnienia doboru kluczowych hiperparametrów. Poniżej przedstawiono szczegółowy opis konfiguracji każdego z modeli wraz z listingami najważniejszych fragmentów kodu.

10.1. Powtarzalność wyników badań

We wszystkich implementowanych modelach oraz funkcjach podziału zbioru danych zastosowano stałe ziarno losowości `random_state=42` (lub `seed=42` dla biblioteki `river`). W uczeniu maszynowym inicjalizacja wag sieci neuronowych, losowanie cech w drzewach decyzyjnych czy mieszanie próbek opierają się na generatorach liczb pseudolosowych. Ustawienie stałego ziarna, w tym przypadku popularnej w środowisku programistycznym liczby 42, jest kluczową praktyką badawczą. Gwarantuje to determinizm eksperymentu, czyli pozwala na dokładne i sprawiedliwe porównanie czasów wykonania oraz metryk F1-Score na różnych platformach sprzętowych, mając pewność, że na każdym urządzeniu zainicjalizowano matematycznie identyczny model.

10.2. Implementacja Autoenkodera

Listing 10.1. Parametry uczenia Autoenkodera

```
# Inicjalizacja modelu
model = MLPRegressor(hidden_layer_sizes=(32, 16, 32), activation='relu', solver='
    adam', max_iter=300, random_state=42)

# Trening (tylko na klasie NORMAL)
mask_normal = (y_train == 0).values
X_train_normal_scaled = X_train_scaled[mask_normal]
model.fit(X_train_normal_scaled, X_train_normal_scaled)
```

Wykorzystano wielowarstwowy perceptron (`MLPRegressor`), ponieważ w trybie uczenia nienadzorowanego wejście sieci (zmienna `X`) jest podawane również jako oczekiwane wyjście (zmienna `y`). Architektura składająca się z warstw o rozmiarach 32, 16 i 32 (10.1) węzłów realizuje fundamentalne założenie autoenkodera, tworząc tzw. "wąskie gardło" (bottleneck) w środkowej warstwie. Wejściowy wektor cech jest kompresowany z `n`-wymiarów początkowych przez warstwę 32 neuronów do zaledwie 16 neuronów, a następnie rekonstruowany z powrotem przez warstwę 32 neuronów do oryginalnego

wymiaru. Taka kompresja zapobiega zjawisku, w którym sieć uczy się zwykłej funkcji tożsamościowej, czyli przepisywania wejścia na wyjście 1:1 i zmusza model do wyeks-
trahowania najważniejszych, ukrytych cech sygnału.

Linijki izolujące klasę `NORMAL` i trenujące model wyłącznie na niej stanowią esencję detekcji anomalii. Gdybyśmy do modelu wprowadzili cały zbiór, łącznie z anoma-
liami, sieć nauczyłaby się rekonstruować również błędy. Ucząc ją tylko poprawnych zachowań maszyny, zapewniamy, że błąd rekonstrukcji MSE wystrzeli w górę, gdy po-
jawi się nieznana anomalia.

10.3. Implementacja Isolation Forest

Listing 10.2. Parametry uczenia Isolation Forest

```
model = IsolationForest(n_estimators=100, n_jobs=-1, random_state=42)
model.fit(X_train_normal_scaled)

# Odwrocenie wyników funkcji decyzyjnej
test_scores = -model.decision_function(X_test_scaled)
```

Liczba 100 (Listing 10.2) określa rozmiar lasu (ilość drzew izolacyjnych). W ory-
ginalnej pracy badawczej dotyczącej tego algorytmu autorzy udowodnili, że długości
ścieżek w drzewach decyzyjnych bardzo szybko zbiegają się do ostatecznych wartości.
Zwiększenie tej liczby np. do 500 czy 1000 drzew drastycznie wydłużyłoby czas obli-
czeń i zużycie pamięci RAM na urządzeniach brzegowych (Raspberry Pi, Jetson), nie
przynosząc zauważalnej poprawy trafności klasyfikacji. Wartość 100 stanowi optymalny
kompromis między wydajnością a szybkością.

Zastosowanie operacji `-model.decision_function()` jest niezwykle ważnym
zabiegiem inżynierskim w kodzie. Domyślnie biblioteka `scikit-learn` zwraca wyniki
ujemne dla próbek anomalnych, a dodatnie dla normalnych. Odwrócenie znaku wartości
ujednolica logikę programu dla wszystkich trzech algorytmów, w każdym przypadku
wprowadzona zostaje zasada “im wyższy wynik punktowy, tym większa anomalia”, co
pozwala na zastosowanie jednej uniwersalnej funkcji wyliczania progów.

10.4. Implementacja Half-Space Trees

Listing 10.3. Parametry uczenia HS-Trees

```
model = anomaly.HalfSpaceTrees(n_trees=25, height=15, window_size=70000, seed=42)

# Petla strumieniowego uczenia online
for i, xi in enumerate(X_train_normal_scaled):
```

```
model.learn_one(dict(enumerate(xi)))
```

Algorytm strumieniowy musi reagować błyskawicznie na pojedyncze zdarzenia. Ponieważ struktura drzew HS-Trees jest aktualizowana i przeszukiwana przy każdej iteracji (dla każdej pojedynczej próbki danych), mniejsza liczba drzew, `n_trees=25`, optymalizuje opóźnienie (latencję) na słabych procesorach typu ARM. Dodatkowo parametr `height=15` narzuca twardy limit na głębokość każdego drzewa (maksymalnie 2^{15} liści na drzewo). Zabezpiecza to moduły IoT, posiadające ograniczoną ilość pamięci operacyjnej, przed niekontrolowanym rozrostem struktur danych w przypadku długotrwałego nasłuchiwanie na dane ze strumienia.

Fundamentalną cechą algorytmów strumieniowych (Online Machine Learning), odróżniającą je od klasycznych modeli wsadowych (batch processing), jest zdolność do ciągłej adaptacji i usuwania nieaktualnych danych. Biblioteka `river` realizuje tę funkcjonalność za pomocą mechanizmu przesuwnego okna (sliding window). Dobór parametru `window_size=70000` w implementacji algorytmu Half-Space Trees nie był przypadkowy i wynikał bezpośrednio z charakterystyki analizowanego zbioru danych. Ponieważ całkowity wolumen użytych danych historycznych wynosił 140 000 rekordów, ustawienie szerokości okna na 70 000 oznacza, że model w każdym momencie ewaluacji przetwarza i opiera swoje decyzje na rozkładzie statystycznym stanowiącym dokładnie połowę całkowitego "cyklu życia" badanej maszyny.

Taka konfiguracja stanowi optymalny kompromis inżynierski. Z jednej strony, utrzymywanie w pamięci aż 70 000 punktów referencyjnych buduje wysoce stabilny profil "normalności". Dzięki temu system wykazuje odpowiednią bezwładność, jest odporny na chwilowe zaszumienia sygnału z sensorów i nie generuje fałszywych alarmów przy niegroźnych mikroskokach. Z drugiej strony, zastosowanie sztywnego limitu bufora zabezpiecza model przed kluczowym w systemach przemysłowych zjawiskiem dryfu koncepcji (concept drift). Układy mechaniczne ulegają naturalnemu zużyciu, a ich domyślne parametry pracy (np. drgania, temperatura) mogą z biegiem czasu powoli ewoluować. Poprzez sukcesywne usuwanie najstarszych obserwacji wykraczających poza połowę cyklu danych, algorytm HS-Trees w sposób płynny i autonomiczny adaptuje się do nowych warunków pracy, nie traktując ich od razu jako krytycznej awarii. Ponadto, narzucenie twardego limitu elementów zapamiętywanych przez struktury drzewiaste gwarantuje stałą i przewidywalną złożoność pamięciową, co chroni wbudo-

wane platformy brzegowe przed wyczerpaniem zasobów RAM, niezależnie od tego, jak długo urządzenie pozostaje uruchomione.

Zastosowanie funkcji `model.learn_one(dict(enumerate(xi)))` uwidacznia różnicę architektoniczną biblioteki `river`. Zamiana wektora NumPy na słownik języka Python w każdej iteracji symuluje odebranie pojedynczego pakietu JSON/słownikowego bezpośrednio z maszyny przemysłowej za pośrednictwem sieci (np. protokołem MQTT), co stanowi doskonałe odwzorowanie produkcyjnego scenariusza w urządzeniach Edge.

10.5. Obliczanie progu anomalii

Listing 10.4. Próg anomalii

```
threshold = np.percentile(train_scores, 98.8)
pred_labels_test = (test_scores > threshold).astype(int)
```

Próg anomalii dla poszczególnych algorytmów był pozyskiwany w sposób następujący:

- a) **Autoencoder** – poprzez funkcję `model.predict(X_train_normal_scaled)` generowana jest odpowiedź modelu jak wyglądają dane wyjściowe na próbach na których model był uczony. Następnie liczony jest średni błąd rekonstrukcji (różnica między danymi wejściowymi w wiśniowymi, to podnosi kwadratu). Błąd został zapisany do zmiennej `train_scores`
- b) **Isolation forest** – funkcja `model.decision_function` zwraca im niższa wartość im krótsza jest ścieżka, aby logika pasowała do reszty kodu dodano na początku tej funkcji minus aby ta logikę odwrócić i aby krótsza ścieżka otrzymywała wyższą wartość. Następnie wartość zapisywana jest do zmiennej `train_scores`
- c) **Half-space tree** – Funkcja `score_one` wylicza wynik na podstawie tego, jak wiele normalnych próbek wylądowało wcześniej w tych samych obszarach co badana próbka, obliczone wartości trafiają do wartości `train_score`.

Następnie liczony jest próg anomalii `threshold`. Używana jest funkcja percentyla, która sortuje wszystkie liczby w tablicy od najmniejszego do największego. Próg zostaje ustawiony w takim miejscu, że 98.8% wszystkich normalnych odczytów znalazło się poniżej niego. Pozostałe 1.2% wyników (najwyższe błędy w zbiorze normalnym) jest traktowanej jako anomalia.

11. Proces klasyfikacji i detekcji na zbiorze testowym

Po ustaleniu granicy decyzyjnej (threshold) na podstawie danych treningowych, system przechodzi do docelowego etapu klasyfikacji na niewidzianym wcześniej zbiorze testowym. Ewaluacja ta przebiega symetrycznie do fazy uczenia. W pierwszej kolejności modele analizują nowe próbki i generują surowe miary odchyłeń, np. błędy rekonstrukcji, długości ścieżek drzew binarnych, które są agregowane w wektorze wynikowym o nazwie `test_scores`.

Następnie implementowana jest właściwa logika detekcyjna. Każda wyestymowana wartość numeryczna z wektora testowego poddawana jest weryfikacji relacyjnej względem wcześniej wyliczonego progu anomalii. Algorytm aplikuje na zbiorze warunków boolowskim, jeśli wynik punktowy danej próbki jest ostro większy niż założony próg, operacja zwraca wartość logiczną `True`. Jeśli natomiast odchylenie jest mniejsze lub równe wartości progowej, układ interpretuje to jako pracę w normie, zwracając `False`.

W celu ujednolicenia formatu danych z matematycznymi standardami ewaluacji (wymaganymi do wyliczenia macierzy pomyłek oraz metryk precyzji), uzyskany wektor logiczny jest natychmiastowo poddawany rzutowaniu typów. Za pomocą wbudowanej metody konwersji (`.astype(int)`), wartości logiczne zamieniane są na reprezentację całkowitoliczbową. W rezultacie powstaje finalna tablica predykcji o nazwie `pred_labels_test`. Składa się ona wyłącznie z wartości binarnych, zera (0) oznaczającego poprawny stan maszyny (NORMAL) oraz jedynki (1) jednoznacznie wskazującej na zidentyfikowaną usterkę (ANOMALIA). Tablica ta stanowi bezpośredni materiał wejściowy do oceny skuteczności klasyfikacyjnej sprzętu.

12. Wyniki detekcji i parametry wydajnościowe

Tabela 12.1. Metryki jakości klasyfikacji i macierz pomyłek

Platforma	Algorytm	Thresh.	Prec.	Rec.	F1	Acc.	TP	FP	TN	FN
PC	Autoencoder	0.0081	0.0287	0.9737	0.0558	0.9621	74	2502	63518	2
PC	Isolation Forest	0.0778	0.0753	0.7895	0.1375	0.9886	60	737	65283	16
PC	Half-Space Trees	0.9762	0.0987	0.8684	0.1772	0.9907	66	603	65417	10
Raspberry Pi	Autoencoder	0.0081	0.0287	0.9737	0.0558	0.9621	74	2502	63518	2
Raspberry Pi	Half-Space Trees	0.9762	0.0987	0.8684	0.1772	0.9907	66	603	65417	10
Raspberry Pi	Isolation Forest	0.0769	0.0756	0.8158	0.1384	0.9883	62	758	65262	14
Jetson	Autoencoder	0.0081	0.0287	0.9737	0.0558	0.9621	74	2502	63518	2
Jetson	Isolation Forest	0.0778	0.0753	0.7895	0.1375	0.9886	60	737	65283	16
Jetson	Half-Space Trees	0.9975	0.6667	0.0263	0.0506	0.9989	2	1	66019	74
<i>ESP - Model nauczony na PC, przepuszczenie przez wagi (Inference)</i>										
ESP	Autoencoder	0.0081	0.0287	0.9737	0.0558	0.9621	74	2502	63518	2
ESP	Isolation Forest	0.0778	0.0753	0.7895	0.1375	0.9886	60	737	65283	16
ESP	Half-Space Trees	0.9762	0.0987	0.8684	0.1772	0.9907	66	603	65417	10

Tabela 12.2. Parametry wydajnościowe i obciążenie zasobów

Platforma	Algorytm	Tren. [s]	Detekcja [s]	Lat. śr. [ms]	P95 [ms]	P99 [ms]	Przepust. [op/s]	CPU [%]	RAM [MB]	Temp. [°C]
PC	Autoencoder	9.855	0.0454	0.0671	0.0785	0.1041	1456109	13.0	3.535	brak
PC	Isolation Forest	1.319	0.1308	4.0783	5.257	5.853	505489	21.6	68.474	brak
PC	Half-Space Trees	134.540	13.8561	0.2080	0.2301	0.2375	4770	9.8	268.981	brak
Raspberry Pi	Autoencoder	18.223	0.1471	0.0844	0.0892	0.1002	449323	25.1	3.544	49.0
Raspberry Pi	Half-Space Trees	233.411	16.2614	0.2460	0.2680	0.2777	4064	23.2	268.981	47.4
Raspberry Pi	Isolation Forest	1.722	0.2364	6.2460	6.350	6.913	279590	25.5	44.001	46.9
Jetson	Autoencoder	27.308	0.3307	0.2419	brak	0.2924	199888	58.7	3.483	brak
Jetson	Isolation Forest	2.815	0.4014	10.9220	brak	11.925	164659	18.5	46.593	brak
Jetson	Half-Space Trees	2684.675	51.8407	0.8049	brak	0.9051	1274	48.8	400.044	brak
<i>ESP - Model nauczony na PC, przepuszczenie przez wagi (Inference)</i>										
ESP	Autoencoder	9.855	0.000062	0.0873	0.1725	0.2009	11450	brak	0.0417	65.0
ESP	Isolation Forest	1.319	0.001072	1.0792	1.4383	1.5156	926	brak	0.0417	64.8
ESP	Half-Space Trees	134.540	0.000272	0.2320	0.3063	0.3245	4310	brak	0.0422	65.0

- wartości chwilowe podstawowych wielkości fizycznych używanych np. w elektro-technice należy zapisać małymi literami, np. u , i , lub stosować zapis np. $u(t)$, lub stosować indeks „ t ” przy wielkości, np. U_t ,

13. Podsumowanie i wnioski końcowe

Literatura